Peekaboo: Improving Deployed Decision Models from Right / Wrong Feedback Alone

Abhijith Neil Abraham Fariz Rahman Fadil Rahman

October 2026

Abstract

Small decision models such as Vega and Laya answer typed questions (pick one option, yes / no, a rating) with calibrated probabilities and run locally in milliseconds. Once deployed they are rarely given the answer key for the decisions they make, but they are constantly told whether a decision was right or wrong: a user accepts or cancels, an alert is confirmed or dismissed. We ask whether a deployed decision model can improve from that one bit per decision, and we answer on the benchmarks the models' own authors publish. Peekaboo trains a small add-on (a gated LoRA adapter on Vega's decision engine; a copy of Laya's decision head, optionally with the encoder) by REINFORCE on right / wrong rewards with a running baseline, kept on a KL leash to the released model, and then restores calibration with a single temperature fitted from right / wrong feedback as well, using only the fact that a calibrated model's chosen answer is right with the probability it states. A feedback-checked switch keeps the add-on only where held-out feedback says it beats the base model. Across six benchmarks (Vega's 43 public tasks, JevBench and DecisionBench; Laya's typed-decisions; emotion and AG News), two Vega sizes, Laya, three seeds and every ablation, Peekaboo beats VegaML's own test-time-training method, which trains on the answer key, on four of six benchmarks at both sizes (DecisionBench: +10.4 and +6.7 points over base against TTT's +2.2 and +0.8), lands within one to two points of an adapter trained on the full answer key, and is the only method that improves Vega 4B on JevBench (+4.5). On Laya, Peekaboo on the head alone gives +13.2 points on typed-decisions where Laya's released answer-key fine-tune of the whole model gives +40.2; unfreezing the encoder under the same right / wrong signal gives ? on typed-decisions and overtakes Laya's own fine-tune on the public tasks (0.572 vs. 0.551) and DecisionBench (0.559 vs. 0.438). Removing the leash or the temperature step costs accuracy or multiplies calibration error two- to four-fold. Every base number reproduces its README value, all code and raw results are released, and one script reruns the whole grid.

1 Introduction

A *decision model* takes a situation and a typed question (which of these options? yes or no? how severe, on a scale?) and returns a probability for each option in one forward pass. Two recent open releases, VegaML's Vega [16] and Convai's Laya [2], target the same deployment: thousands of small decisions a day inside agents, moderation pipelines and support systems, on local hardware, with probabilities that can be trusted enough to route the uncertain cases to a human.

Both come with a way to improve on a user's task, and both ways need the answer key. Vega offers test-time training (TTT): a per-task classifier fitted to its frozen features from labelled rows, and adapters trained the same way. Laya's recipe is to fine-tune the whole model on labelled rows with its RLCD objective; its released `laya-typed-decisions` checkpoint is the result on its headline benchmark. But a deployed decision model seldom collects labels. What it collects is *feedback on its own answers*: the suggested tool call was cancelled, the flagged message was not toxic after all, the escalated ticket really did need escalation. Each

such event says whether the model was right, not what the right answer was. In the language of contextual bandits [12, 13], the model observes the reward of the action it took and nothing about the others.

This paper asks a narrow question and answers it on the benchmarks the models' authors use themselves: **can a deployed decision model improve from right / wrong feedback alone, and how does that compare with the model's own answer-key method?** Our method, Peekaboo[1], has four parts (Section 4):

1. an **add-on** rather than a change to the released model: a rank-8 gated LoRA adapter on the input layers of Vega's decision engine, or a trainable copy of Laya's two-layer decision head (optionally with a copy of the encoder);
2. **REINFORCE** [27] on the right / wrong reward of a sampled answer, with a running-mean baseline;
3. a **KL leash** to the released model's probabilities, which stops the collapse that policy-gradient training from one bit per row otherwise produces;
4. a **feedback-fitted temperature** (the "honesty fix"): the single temperature under which the right / wrong record on held-out rows is most likely, using the fact that a calibrated model's sampled answer is right exactly with its stated probability.

A fifth, optional part is a **feedback-checked switch** that keeps the add-on only where it beats the base model on held-out right / wrong feedback.

We evaluate on six benchmarks: the three in Vega's README (43 public tasks, JevBench [23], DecisionBench [7]), the one in Laya's README (typed-decisions [14]), and two standard text classification sets with large feedback pools (emotion [21] and AG News [28]). We run Vega 0.8B, Vega 4B and Laya, each with its own answer-key improvement, an answer-key version of our add-on as an upper bound, Peekaboo, and every ablation (no leash, no honesty fix, switch on / off, one add-on per benchmark vs. one pooled add-on), three seeds each: 233 steps in all. The metrics are computed with VegaML's own vendored benchmark code, and every base model reproduces its published number.

Findings. (i) Against Vega's own improvement, Peekaboo wins on the public tasks, DecisionBench, typed-decisions and AG News at both sizes with the weaker signal; TTT wins on emotion. (ii) Peekaboo is within one to two points of training the same adapter on the full answer key. (iii) It is the only method that improves Vega 4B on JevBench, and pooling the feedback of all benchmarks into one add-on removes the 0.8B's JevBench loss. (iv) On Laya, Peekaboo on the decision head is far behind Laya's whole-model answer-key fine-tune on typed-decisions (+13.2 vs. +40.2); with the encoder unfrozen it reaches ? there and overtakes the fine-tune on the public tasks (0.572 vs. 0.551) and DecisionBench (0.559 vs. 0.438), while the fine-tune loses 5.2 points on JevBench. (v) The leash and the temperature step are both necessary: without the leash accuracy drops on five of six benchmarks; without the temperature step accuracy is unchanged but calibration error is two to four times higher.

2 Related work

Learning from bandit feedback. Learning a structured predictor from the reward of its own output goes back to bandit structured prediction [22] and was shown to work with real human ratings [10]. Off-policy variants learn from logged feedback with importance weights [3, 25]. Peekaboo is the on-policy case, REINFORCE with a baseline [27], applied to a model that already has a strong zero-shot policy; our no-engine control (Section 6) shows that this starting point, not the update rule, is what makes one bit per row enough.

KL-regularised fine-tuning. A KL penalty to a reference policy is standard in RLHF [1, 17] and in offline RL for dialogue [9]; DPO [20] and KTO [5] fold it into a supervised objective, KTO from binary desirable

[1]The model only gets to *peek* at whether it was right; it never sees the answer key.

/ undesirable labels. Our leash plays the same role in a much smaller setting, and we measure what it buys: removing it costs accuracy on five of six benchmarks (Section 6.3).

Calibration. Temperature scaling [6, 18] fits one scalar on labelled held-out data. Our honesty fix is temperature scaling whose likelihood is written over right / wrong outcomes of sampled answers instead of labels: for a calibrated model, $\Pr[\text{right} \mid a] = p(a)$ for any answer a, so each (answer, outcome) pair is a Bernoulli observation of $p_T(a)$. We measure calibration with the expected calibration error [15] as implemented in VegaML's benchmark code.

Adapting small decision models. Test-time training [24] fits a model to the test distribution; VegaML's `fit()` fits a per-task classifier (PCA then logistic regression) on labelled rows and keeps it when it beats chance in cross-validation. LoRA [8] is the form our Vega add-on takes, using VegaML's own gated adapter mechanism. Laya's RLCD trains the whole encoder-plus-head on labels with a perturbation policy gradient on proper scoring rules.

3 The two decision models

Vega. Vega [16] reads the situation, question and options with a frozen Qwen3.5 backbone [19] (0.8B or 4B parameters), pools hidden states from two intermediate layers, and hands them to a small "physics engine" (14.3M parameters at 0.8B, 30.9M at 4B): each option is an energy well in a 64-dimensional landscape, a particle is dropped and settles, and the probabilities are $P(a) = \text{softmax}(-E_a/\tau(s))$ with a per-question temperature read off the particle's trajectory. Only the engine and its gated adapters were trained; the backbone never was. Vega's own ways to improve on a task are the TTT head and adapters, both trained from labelled rows.

Laya. Laya [2] is a ModernBERT-large encoder [26] (421M parameters, trained as part of Laya) whose input contains the situation, the question and every option, each option marked with a special token. A two-layer transformer decision head scores each marker position; the scores are divided by a temperature chosen per (question type, option-count bucket) and softmaxed. It was trained with RLCD. Laya's way to improve on a task is to fine-tune the whole model on labelled rows; `laya-typed-decisions` is that fine-tune on typed-decisions' training split.

Both models expose the same interface (a typed question, a probability per option), which is what lets one method and one evaluation harness serve both.

4 Peekaboo

4.1 Setting

A decision model is a policy $\pi_0(a \mid s)$ over the options a of a typed question s. During deployment the model answers a stream of questions. For each one it (or the system around it) commits to an answer a, and later a reward $r \in \{0, 1\}$ arrives: right or wrong. The correct option is never revealed. From a pool of such (question, answer, reward) records we want a better policy π that keeps calibrated probabilities. In our experiments the reward is computed from the benchmark's gold answer and the answer is sampled from the current policy, so feedback is on-policy and noise-free; Section 8 discusses this.

4.2 The add-on

We never change the released model. On Vega we attach one gated LoRA adapter [8] of rank 8 to the input layers of the engine with VegaML's own `attach_adapter`: 409k parameters at 0.8B, 857k at 4B, i.e. 2.9% and 2.8% of the engine; its gate starts open and the shipped adapters keep working. The policy is Vega's own calibrated output with the adapted energies and the shipped per-question temperature. On Laya we make a trainable copy of the decision head (the type embedding, the two transformer layers and the scorer); the encoder is frozen and run once per row. In the *encoder-unfrozen* variant we also copy the encoder and train it at Laya's own encoder learning rate (2.5×10^{-5}) in micro-batches of 8, so that Peekaboo changes as much of Laya as Laya's own fine-tune does.

4.3 The update

For a batch of questions the policy samples one answer per question, $a \sim \pi(\cdot \mid s)$, receives $r = \mathbf{1}[a \text{ is right}]$, and minimises

$$\mathcal{L} \;=\; -\,(r - b)\,\log \pi(a \mid s) \;+\; \beta\,\mathrm{KL}\big(\pi(\cdot \mid s)\,\|\,\pi_0(\cdot \mid s)\big), \qquad \beta = 0.1, \tag{1}$$

where b is an exponential running mean of recent rewards ($b \leftarrow 0.9\,b + 0.1\,\bar{r}$, initialised at 0.5). The first term is REINFORCE with a baseline: an answer rewarded above the running average becomes more likely in similar situations, one rewarded below it less likely. The second term is the leash. Without it, one bit per row drives the policy to collapse onto a few options (NLL rose to 7.5 in our pilot runs; Section 6.3 measures the cost on every benchmark). Training runs for 8 passes over the feedback rows in batches of 32 with Adam at learning rate 10^{-3} (Vega adapter) or 10^{-4} (Laya head; Laya's own head rate) and gradient-norm clipping at 1.

4.4 The honesty fix: a temperature from right / wrong

Policy-gradient training sharpens the probabilities: before correction the trained add-ons have ECE between 0.17 and 0.35 (Table 6). Standard temperature scaling [6] would fit a scalar T on labelled rows. We have no labels, but we have the following: if a model is calibrated, the answer a it samples is right with probability exactly $p(a)$. So on a held-out calibration split (20% of the feedback rows of each task) we sample $a \sim \pi$, observe r, and choose

$$T^{\star} = \arg\max_{T} \sum_{i} \Big[r_i \log p_T(a_i \mid s_i) + (1 - r_i) \log \big(1 - p_T(a_i \mid s_i)\big) \Big], \qquad p_T = \mathrm{softmax}(\mathrm{logits}/T), \tag{2}$$

over a grid of 61 values of T between 0.25 and 32. The fitted temperatures lie between about 3 and 9 (Figure 5), confirming that the trained policy was far too sharp. The same identity holds for answers chosen by anyone, so $T^{\star}$ can also be fitted from logged feedback on the released model's answers.

4.5 The switch

A deployment that cannot afford a regression wants an add-on that is on only where it helps. On the same calibration split we sample one answer from the adapted policy and one from the released model per row, rate both right / wrong, and keep the add-on only if it was right more often. This costs nothing beyond the feedback already collected. We report every Peekaboo result with and without the switch.

Table 1: The six benchmarks. "feedback rows" are the rows whose right / wrong signal Peekaboo may learn from (80%) and calibrate on (20%); no test row is ever used for learning. Question types: pick one (choice), yes / no, rating scale (score). JevBench has no training split, so Peekaboo learns by 5-fold cross-fitting with paraphrase groups kept in one fold.

benchmark	reported by	test rows	feedback rows	what it contains	types (test)
43 public tasks	Vega	2,535	2,134	43 public datasets as typed questions, up to 60 test rows each: intent (77 and 150 options), topic, safety, sentiment, entailment, science and medical QA, law, reading, code; macro-averaged over tasks	1,709 / 707 / 119
JevBench	Vega	231	(5-fold)	rules, policies and judgment calls in 18 families; tiers hard 111 / original 72 / easy 48; paraphrase groups	139 / 74 / 18
DecisionBench	Vega	1,075	3,205	43 tasks in 28 domains (agent control, coding agents, finance, law, privacy, robotics, …); candidate selection 750, binary 225, ordinal 100 (asked as pick-one); up to 255 options	913 / 162 / 0
typed-decisions	Laya	2,000	6,000	400 test and 1,200 training JSON cases (tickets, invoices, agent traces, security alerts), five decisions each, four workflows of 500 test decisions	600 / 600 / 800
emotion	(ours)	1,503	4,000	six emotions of a tweet	1,503 / 0 / 0
AG News	(ours)	1,912	4,000	four news sections	1,912 / 0 / 0

4.6 Pooled Peekaboo

The default trains one add-on per benchmark from that benchmark's feedback rows. *Pooled* Peekaboo trains one add-on per model from the feedback rows of all five benchmarks that have a training split and evaluates it on all six; JevBench, which has no training split, is pure transfer for the pooled add-on.

4.7 Cost

The backbone is untouched, so inference cost is unchanged apart from the add-on: on a laptop CPU, one row at a time, Vega's engine takes 6.3 ms (p50) released and 6.5 ms with the adapter at 0.8B, 11.4 vs. 12.3 ms at 4B, against about 75 to 245 ms for the backbone on a T4 GPU. Training on cached features takes one to fifteen minutes per benchmark and seed on one L40S GPU; the Laya encoder-unfrozen runs take 4 to 15 minutes each.

5 Experimental setup

5.1 Benchmarks

We use exactly the benchmarks the two READMEs report, plus two classification sets chosen for their large pools of feedback rows (Table 1, Figure 1). Rows are read, asked and scored with VegaML's benchmark harness, vendored unmodified (Apache-2.0), so every number is comparable with Vega's README.

For DecisionBench we use the README's 1,075-row slice (25 rows per task, the notebook's seed) as the test set and a 100-per-task superset of the same tasks for feedback, as in our earlier DecisionBench

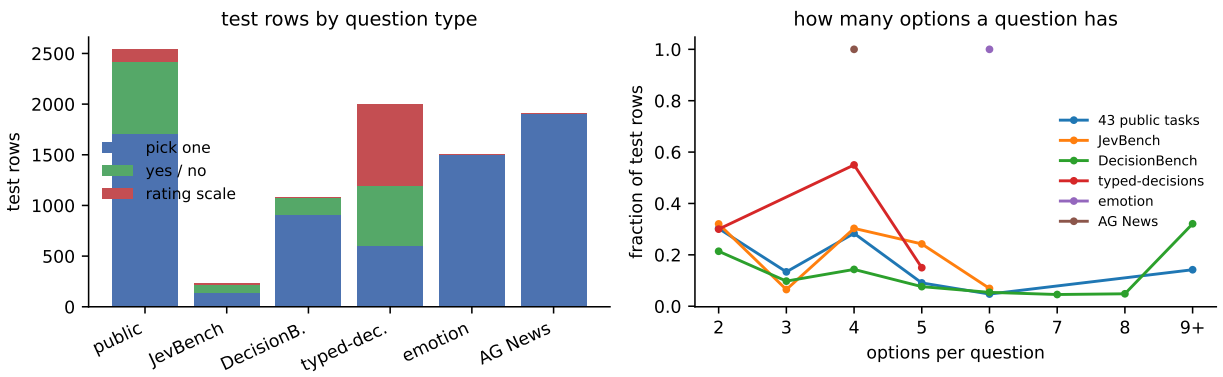


Figure 1: Test-row distribution of the six benchmarks: question types (left) and number of options per question (right). DecisionBench and the public menu include questions with dozens to hundreds of options.

study. For the public menu we use the harness's 60 evaluation rows per task and its 50 training rows per task as the feedback pool, the same 50 rows VegaML's TTT trains on. For typed-decisions we use the public LocalLLaMA release, which is what Laya reports on; Vega's README also quotes a typed-decisions number on six unpublished workflows that cannot be rerun.

5.2 Protocol

Every method learns only from feedback (or label) rows and is scored on the test rows; on JevBench, 5-fold cross-fitting keeps all paraphrases of a question in the same fold. The feedback rows of each task are split 80 / 20 into training and calibration rows with a fixed hash. All stochastic methods are run with seeds 0, 1, 2; we report mean ± s.d. over seeds. Gains are paired: the per-row correctness of a method (averaged over its seeds) minus that of the base model on the same rows, with a 95% bootstrap CI over rows [4], resampled within task for the macro-averaged public menu. Accuracy, ECE (15 equal-width bins, as in the README), NLL, skill, macro-F1, the JevBench tiers and paraphrase consistency, DecisionBench's domain slices and typed-decisions' soft accuracy all come from the harness's `score_metrics`.

5.3 Baselines

- **Base model**: Vega 0.8B, Vega 4B, Laya, as released. Each is first *replayed*: we re-run the frozen model on the cached inputs and check that its probabilities match the official prediction call (max absolute difference $\leq 5 \times 10^{-5}$ for Laya, 0 for Vega), and that its scores match the README.
- **Their improvement, answer key**: for Vega, VegaML's TTT head (`fit()`, default settings) trained on the labelled feedback rows of each task; for Laya, the released `laya-typed-decisions` checkpoint, Laya's whole-model fine-tune on typed-decisions' labelled training split, scored on every benchmark.
- **Answer-key add-on**: the same adapter (Vega) or head copy (Laya) trained with cross-entropy on the labels of the same rows and the same KL leash, then standard label-based temperature scaling on the same calibration rows. This is the upper bound Peekaboo is measured against: same capacity, full information.
- **Peekaboo** and its ablations: no leash ($\beta = 0$), no honesty fix (the trained add-on with $T = 1$), with the switch, pooled, and on Laya the encoder-unfrozen variant.
- **No-engine control** (DecisionBench study): a bilinear scorer on the same frozen features with the same feedback, leash and calibration, without Vega's engine.

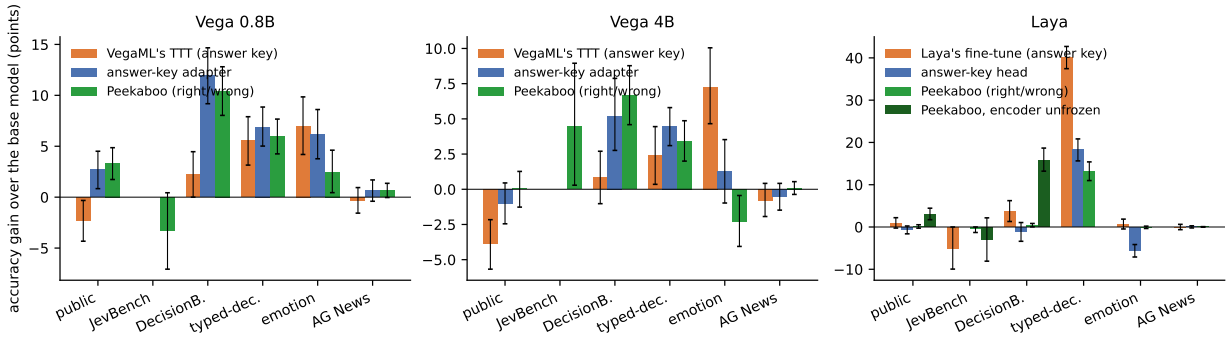


Figure 2: Accuracy gain over each base model on the six benchmarks (points; error bars are paired 95% bootstrap CIs). Orange: the model's own answer-key improvement (VegaML's TTT head; Laya's released typed-decisions fine-tune). Blue: our add-on trained on the answer key. Greens: Peekaboo, from right / wrong only.

Table 2: Accuracy. Mean over 3 seeds (± s.d.) where a method is stochastic. JevBench has no training split: Peekaboo learns by 5-fold cross-fitting (paraphrase groups kept together), pooled Peekaboo never sees it, and VegaML's TTT and the answer-key methods are not defined for it.

model	method	learns from	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
Vega 0.8B	base model	none	0.556	0.550	0.509	0.665	0.568	0.901
	VegaML's TTT head	answer key	0.533	·	0.531	0.721	0.638	0.897
	answer-key adapter	answer key	0.584 ±0.005	·	0.628 ±0.008	0.734 ±0.000	0.630 ±0.011	0.907 ±0.001
	Peekaboo	right / wrong	0.590 ±0.013	0.517 ±0.018	0.613 ±0.011	0.725 ±0.012	0.593 ±0.009	0.907 ±0.002
	pooled Peekaboo	right / wrong	0.596 ±0.006	0.548 ±0.011	0.606 ±0.011	0.720 ±0.015	0.567 ±0.040	0.911 ±0.005
Vega 4B	base model	none	0.715	0.667	0.631	0.730	0.579	0.927
	VegaML's TTT head	answer key	0.677	·	0.639	0.753	0.651	0.919
	answer-key adapter	answer key	0.705 ±0.005	·	0.683 ±0.011	0.774 ±0.006	0.592 ±0.022	0.921 ±0.003
	Peekaboo	right / wrong	0.716 ±0.007	0.711 ±0.004	0.697 ±0.006	0.764 ±0.006	0.556 ±0.009	0.928 ±0.002
	pooled Peekaboo	right / wrong	0.720 ±0.005	0.687 ±0.002	0.682 ±0.008	0.744 ±0.018	0.544 ±0.009	0.932 ±0.003
Laya	base model	none	0.542	0.589	0.401	0.361	0.597	0.930
	laya-typed-decisions (Laya's fine-tune)	answer key	0.551	0.537	0.438	0.762	0.604	0.930
	answer-key head	answer key	0.535 ±0.004	·	0.389 ±0.009	0.544 ±0.005	0.540 ±0.032	0.930 ±0.000
	Peekaboo, head only	right / wrong	0.543 ±0.001	0.584 ±0.000	0.405 ±0.002	0.493 ±0.008	0.596 ±0.001	0.930 ±0.000
	Peekaboo, encoder unfrozen	right / wrong	0.572 ±0.007	0.558 ±0.011	0.559 ±0.005	·	·	·
	pooled Peekaboo, head only	right / wrong	0.538 ±0.002	0.569 ±0.007	0.398 ±0.006	0.475 ±0.004	0.594 ±0.002	0.930 ±0.000

5.4 Compute and reproducibility

The grid (6 benchmarks × 3 models × methods × 3 seeds, 233 steps) ran on one NVIDIA L40S in about five GPU-hours, from cached backbone features. One script, `experiments/peekaboo_remaining/run.py`, runs every step, skips finished ones, retries failures and writes the tables; the results, logs and the table in this paper are in the repository.[2] All of Section 6.6 ran on a laptop CPU and was reproduced independently on a second machine (Appendix A).

6 Results

6.1 Main comparison

Table 3 and Figure 2 give the headline comparison; Table 2 the absolute accuracies and Table 4 the calibration errors. Three things stand out.

[2] https://github.com/abhijithneilabraham/peekaboo

Table 3: Gain over the base model, in accuracy points. Mean over 3 seeds (± s.d.) where a method is stochastic. Each gain is paired on the same rows against the model's own base; the CIs are in Table 5 and Figure 2. JevBench has no training split: Peekaboo learns by 5-fold cross-fitting (paraphrase groups kept together), pooled Peekaboo never sees it, and VegaML's TTT and the answer-key methods are not defined for it.

model	method	learns from	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
Vega 0.8B	base model	none	–	–	–	–	–	–
	VegaML's TTT head	answer key	-2.3	·	+2.2	+5.5	+7.0	-0.3
	answer-key adapter	answer key	+2.7	·	+11.9	+6.9	+6.1	+0.6
	Peekaboo	right / wrong	+3.4	-3.3	+10.4	+5.9	+2.5	+0.7
	pooled Peekaboo	right / wrong	+4.0	-0.1	+9.7	+5.4	-0.2	+1.1
Vega 4B	base model	none	–	–	–	–	–	–
	VegaML's TTT head	answer key	-3.9	·	+0.8	+2.4	+7.3	-0.8
	answer-key adapter	answer key	-1.0	·	+5.2	+4.5	+1.3	-0.5
	Peekaboo	right / wrong	+0.1	+4.5	+6.7	+3.4	-2.3	+0.1
	pooled Peekaboo	right / wrong	+0.4	+2.0	+5.1	+1.4	-3.5	+0.5
Laya	base model	none	–	–	–	–	–	–
	laya-typed-decisions (Laya's fine-tune)	answer key	+0.9	-5.2	+3.7	+40.2	+0.7	+0.0
	answer-key head	answer key	-0.7	·	-1.2	+18.3	-5.6	+0.0
	Peekaboo, head only	right / wrong	+0.1	-0.4	+0.4	+13.2	-0.1	+0.0
	Peekaboo, encoder unfrozen	right / wrong	+3.0	-3.0	+15.8	·	·	·
	pooled Peekaboo, head only	right / wrong	-0.4	-2.0	-0.2	+11.4	-0.2	+0.0

Table 4: Expected calibration error (lower is better). Mean over 3 seeds (± s.d.) where a method is stochastic. JevBench has no training split: Peekaboo learns by 5-fold cross-fitting (paraphrase groups kept together), pooled Peekaboo never sees it, and VegaML's TTT and the answer-key methods are not defined for it.

model	method	learns from	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
Vega 0.8B	base model	none	0.154	0.173	0.105	0.063	0.033	0.021
	VegaML's TTT head	answer key	0.208	·	0.113	0.139	0.039	0.043
	answer-key adapter	answer key	0.182 ±0.003	·	0.071 ±0.014	0.079 ±0.008	0.059 ±0.013	0.022 ±0.004
	Peekaboo	right / wrong	0.203 ±0.007	0.092 ±0.018	0.055 ±0.015	0.077 ±0.019	0.070 ±0.025	0.016 ±0.004
	pooled Peekaboo	right / wrong	0.193 ±0.024	0.108 ±0.034	0.066 ±0.009	0.062 ±0.031	0.068 ±0.023	0.019 ±0.005
Vega 4B	base model	none	0.141	0.158	0.099	0.050	0.054	0.027
	VegaML's TTT head	answer key	0.173	·	0.112	0.119	0.043	0.041
	answer-key adapter	answer key	0.181 ±0.006	·	0.051 ±0.009	0.056 ±0.007	0.038 ±0.010	0.014 ±0.001
	Peekaboo	right / wrong	0.172 ±0.003	0.098 ±0.008	0.049 ±0.009	0.044 ±0.013	0.035 ±0.012	0.018 ±0.003
	pooled Peekaboo	right / wrong	0.176 ±0.015	0.134 ±0.038	0.057 ±0.019	0.050 ±0.005	0.044 ±0.009	0.016 ±0.006
Laya	base model	none	0.212	0.113	0.174	0.183	0.306	0.012
	laya-typed-decisions (Laya's fine-tune)	answer key	0.166	0.101	0.045	0.206	0.185	0.106
	answer-key head	answer key	0.143 ±0.001	·	0.066 ±0.006	0.035 ±0.008	0.072 ±0.016	0.010 ±0.002
	Peekaboo, head only	right / wrong	0.157 ±0.004	0.063 ±0.012	0.071 ±0.006	0.052 ±0.005	0.086 ±0.014	0.018 ±0.004
	Peekaboo, encoder unfrozen	right / wrong	0.152 ±0.006	0.087 ±0.012	0.062 ±0.009	·	·	·
	pooled Peekaboo, head only	right / wrong	0.164 ±0.007	0.096 ±0.019	0.067 ±0.015	0.052 ±0.016	0.087 ±0.006	0.016 ±0.003

Peekaboo beats Vega's own answer-key method on four of six benchmarks, at both sizes. On DecisionBench, Peekaboo gains +10.4 [+8.0, +12.8] at 0.8B and +6.7 [+4.6, +8.8] at 4B, where VegaML's TTT head gains +2.2 and +0.8 with the labels. On the public menu Peekaboo gains +3.4 and +0.1 while TTT *loses* 2.3 and 3.9 points: the per-task head is fitted to 50 labelled rows and overturns Vega's zero-shot answer on tasks where it should not. On typed-decisions Peekaboo and TTT are close at 0.8B (+5.9 vs. +5.5) and Peekaboo is ahead at 4B (+3.4 vs. +2.4); on AG News both are within a point of base. TTT wins clearly on emotion (+7.0 / +7.3 vs. +2.5 / −2.3): a six-way single-label task with 4,000 labelled rows is exactly where a logistic-regression head on good features shines, and one bit per row is a poor substitute for the full label there.

Table 5: Peekaboo head to head, in accuracy points with a paired 95% bootstrap CI. Positive means Peekaboo is better. "their improvement" is VegaML's TTT head for Vega and the released laya-typed-decisions fine-tune for Laya, both trained with the answer key.

model	benchmark	vs. base	vs. their improvement	vs. answer-key adapter
Vega 0.8B	43 public tasks	+3.4 [+1.7, +4.9]	+5.7 [+3.8, +7.5]	+0.6 [-0.6, +1.9]
	JevBench	-3.3 [-7.1, +0.4]	–	–
	DecisionBench	+10.4 [+8.0, +12.8]	+8.2 [+5.8, +10.7]	-1.5 [-3.4, +0.2]
	typed-decisions	+5.9 [+4.2, +7.7]	+0.4 [-1.5, +2.5]	-0.9 [-1.9, +0.1]
	emotion	+2.5 [+0.4, +4.6]	-4.5 [-6.6, -2.3]	-3.7 [-5.3, -2.1]
	AG News	+0.7 [-0.0, +1.4]	+1.0 [-0.1, +2.1]	+0.0 [-0.8, +0.8]
Vega 4B	43 public tasks	+0.1 [-1.3, +1.3]	+3.9 [+2.2, +5.6]	+1.1 [+0.0, +2.1]
	JevBench	+4.5 [+0.3, +9.0]	–	–
	DecisionBench	+6.7 [+4.6, +8.8]	+5.8 [+3.7, +7.8]	+1.5 [-0.2, +3.1]
	typed-decisions	+3.4 [+2.0, +4.9]	+1.0 [-0.7, +2.7]	-1.1 [-1.9, -0.2]
	emotion	-2.3 [-4.1, -0.4]	-9.5 [-11.6, -7.3]	-3.5 [-5.4, -1.7]
	AG News	+0.1 [-0.4, +0.5]	+0.9 [-0.1, +1.9]	+0.6 [-0.2, +1.4]
Laya, head only	43 public tasks	+0.1 [-0.3, +0.5]	-0.8 [-2.1, +0.5]	+0.8 [+0.0, +1.7]
	JevBench	-0.4 [-1.3, +0.0]	+4.8 [-0.4, +9.5]	–
	DecisionBench	+0.4 [+0.0, +0.8]	-3.3 [-5.8, -0.9]	+1.6 [-0.6, +3.7]
	typed-decisions	+13.2 [+11.0, +15.4]	-26.9 [-28.9, -24.8]	-5.1 [-7.0, -3.4]
	emotion	-0.1 [-0.4, +0.2]	-0.7 [-1.9, +0.4]	+5.6 [+4.1, +7.0]
	AG News	+0.0 [-0.0, +0.1]	+0.0 [-0.6, +0.6]	+0.0 [-0.2, +0.2]
Laya, encoder unfrozen	43 public tasks	+3.0 [+1.7, +4.4]	+2.1 [+0.7, +3.6]	–
	JevBench	-3.0 [-8.1, +2.2]	+2.2 [-3.3, +7.5]	–
	DecisionBench	+15.8 [+13.2, +18.7]	+12.1 [+9.6, +14.8]	–

Peekaboo is within one to two points of training the same add-on on the answer key. The answer-key adapter is the information-rich upper bound with identical capacity. The paired difference Peekaboo − adapter is +0.7 [−0.9, +2.2] (public), −1.5 [−3.5, +0.5] (DecisionBench), −0.9 [−2.5, +0.6] (typed-decisions) and +0.1 (AG News) at 0.8B, and +1.1, +1.5, −1.1, +0.6 at 4B (Table 5); only emotion shows a clear gap (−3.6 / −3.6). The cheap signal buys most of what the expensive one does.

JevBench. JevBench is small (231 rows), has no training split and asks for judgment under paraphrase. Peekaboo is the only method defined on it besides the base model (TTT and the answer-key methods need labelled training rows; the cross-fitted Peekaboo uses rewards on the other folds). At 4B it is the only improvement we found: +4.5 [+0.3, +9.0]. At 0.8B it loses 3.3 points [−7.1, +0.4]; the switch (Section 6.3) reduces that to −0.6 and pooling (Section 6.4) to −0.1.

6.2 Calibration

Table 4 and Figure 3 show the calibration side. Policy-gradient training over-sharpens: without the honesty fix the add-on's ECE is 0.24 (DecisionBench, 0.8B), 0.22 (4B), 0.20 and 0.17 (typed-decisions). The feedback-fitted temperature brings these to 0.055, 0.049, 0.077 and 0.044, below the base model's 0.105, 0.099, 0.063 and 0.050 on DecisionBench and comparable on typed-decisions, and below the answer-key adapter with label-based temperature scaling on DecisionBench (0.071, 0.051). The exception is the public menu, where every trained method, Peekaboo included, is worse calibrated than base Vega (0.203 vs. 0.154 at 0.8B): one temperature per benchmark is too coarse for 43 heterogeneous tasks whose option counts range from 2 to

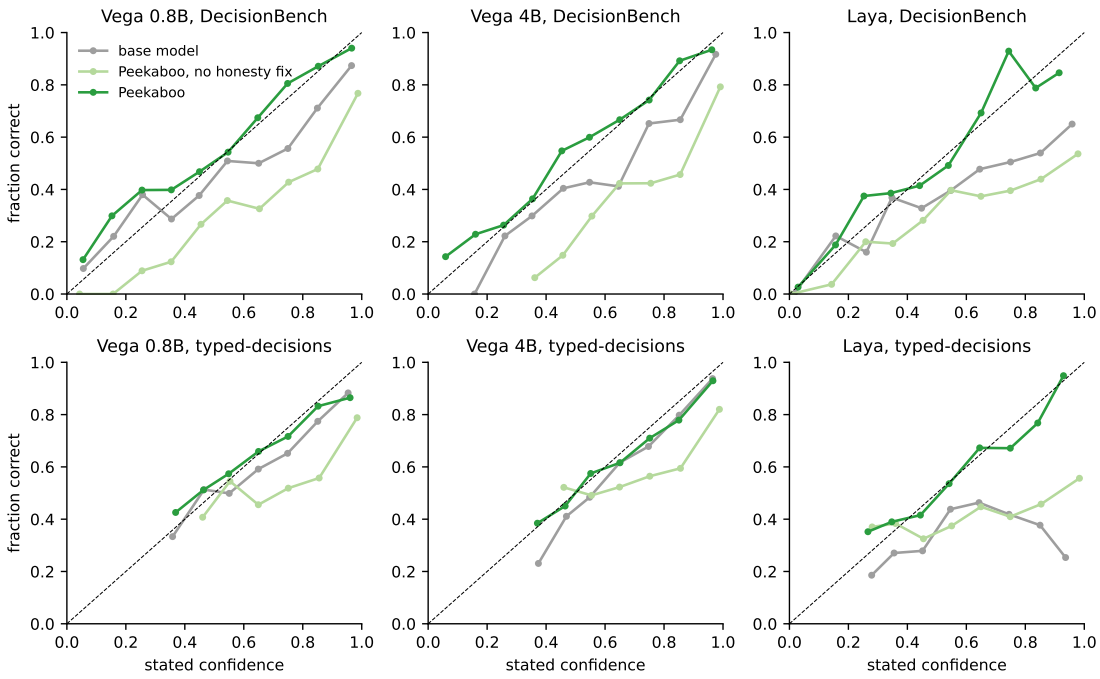


Figure 3: Reliability diagrams (10 bins on the confidence of the predicted option) on DecisionBench and typed-decisions. Grey: base model. Light green: the trained Peekaboo add-on before the honesty fix, always over-confident. Dark green: Peekaboo. The temperature fitted from right / wrong feedback puts the curves back on the diagonal.

151. On Laya the honesty fix is what makes the trained head usable at all: base Laya is poorly calibrated on DecisionBench (0.174) and emotion (0.306), and Peekaboo brings both to 0.07–0.09.

6.3 Ablations

Figure 4 and Table 6 isolate each part.

The leash. With $\beta = 0$ the gain drops on five of the six benchmarks at both sizes: at 0.8B from +3.4 to +1.2 (public), −3.3 to −4.0 (JevBench), +5.9 to +3.3 (typed-decisions) and +2.5 to −3.9 (emotion); at 4B from +6.7 to +5.0 (DecisionBench), +3.4 to +1.1 (typed-decisions) and −2.3 to −5.8 (emotion). Only AG News, where the base model is already at 0.90–0.93, is unaffected. The leash also keeps calibration closer to base on the public menu. In our earlier pilot a stronger leash ($\beta = 1$) was stable but kept only a third of the gain; $\beta = 0.1$ was chosen then and never retuned.

The honesty fix. By construction a temperature does not change the argmax, so accuracy is identical with and without it (Table 6); ECE is 1.6 to 4.4 times higher without it (public 0.203 → 0.328; DecisionBench 0.055 → 0.241; emotion 0.070 → 0.251 at 0.8B). Any system that uses the probabilities, for routing to a human or for conformal sets, needs this step.

The switch. The switch is a safety net, not a booster. It never turns a loss into a gain beyond base and sometimes removes a gain: at 4B it turns DecisionBench's +6.7 into +4.5 and JevBench's +4.5 into +0.9, because on a 42-row calibration fold a sampled comparison is noisy. Where Peekaboo hurts, it helps: JevBench at 0.8B goes from −3.3 to −0.6 and the pooled add-on's emotion loss at 4B from −3.5 to −0.8. Figure 5 shows

Table 6: Every variant and ablation: accuracy (top) and expected calibration error (bottom), mean over 3 seeds. "switch": the add-on is used only where held-out right / wrong feedback says it beats the base model. "no honesty fix": the trained add-on without the feedback-fitted temperature. "KL 0": no leash.

model	variant	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
accuracy							
Vega 0.8B	base model	0.556	0.550	0.509	0.665	0.568	0.901
	Peekaboo	0.590 ±0.013	0.517 ±0.018	0.613 ±0.011	0.725 ±0.012	0.593 ±0.009	0.907 ±0.002
	Peekaboo + switch	0.578 ±0.020	0.544 ±0.004	0.613 ±0.011	0.711 ±0.032	0.593 ±0.009	0.907 ±0.002
	Peekaboo without the honesty fix	0.590 ±0.013	0.517 ±0.018	0.613 ±0.011	0.725 ±0.012	0.593 ±0.009	0.907 ±0.002
	Peekaboo without the leash (KL 0)	0.568 ±0.021	0.509 ±0.025	0.612 ±0.007	0.698 ±0.020	0.529 ±0.020	0.908 ±0.001
	pooled Peekaboo (one add-on, all suites' feedback)	0.596 ±0.006	0.548 ±0.011	0.606 ±0.011	0.720 ±0.015	0.567 ±0.040	0.911 ±0.005
	pooled Peekaboo + switch	0.568 ±0.016	0.548 ±0.011	0.606 ±0.011	0.708 ±0.031	0.567 ±0.040	0.911 ±0.005
	pooled Peekaboo without the honesty fix	0.596 ±0.006	0.548 ±0.011	0.606 ±0.011	0.720 ±0.015	0.567 ±0.040	0.911 ±0.005
Vega 4B	base model	0.715	0.667	0.631	0.730	0.579	0.927
	Peekaboo	0.716 ±0.007	0.711 ±0.004	0.697 ±0.006	0.764 ±0.006	0.556 ±0.009	0.928 ±0.002
	Peekaboo + switch	0.713 ±0.003	0.675 ±0.006	0.676 ±0.032	0.764 ±0.006	0.556 ±0.009	0.927 ±0.000
	Peekaboo without the honesty fix	0.716 ±0.007	0.711 ±0.004	0.697 ±0.006	0.764 ±0.006	0.556 ±0.009	0.928 ±0.002
	Peekaboo without the leash (KL 0)	0.707 ±0.008	0.704 ±0.011	0.681 ±0.002	0.741 ±0.007	0.521 ±0.022	0.930 ±0.002
	pooled Peekaboo (one add-on, all suites' feedback)	0.720 ±0.005	0.687 ±0.002	0.682 ±0.008	0.744 ±0.018	0.544 ±0.009	0.932 ±0.003
	pooled Peekaboo + switch	0.715 ±0.000	0.687 ±0.002	0.651 ±0.029	0.741 ±0.017	0.571 ±0.011	0.927 ±0.000
	pooled Peekaboo without the honesty fix	0.720 ±0.005	0.687 ±0.002	0.682 ±0.008	0.744 ±0.018	0.544 ±0.009	0.932 ±0.003
Laya	base model	0.542	0.589	0.401	0.361	0.597	0.930
	Peekaboo	0.543 ±0.001	0.584 ±0.000	0.405 ±0.002	0.493 ±0.008	0.596 ±0.001	0.930 ±0.000
	Peekaboo + switch	0.542 ±0.000	0.586 ±0.002	0.401 ±0.000	0.493 ±0.008	0.597 ±0.000	0.930 ±0.000
	Peekaboo without the honesty fix	0.543 ±0.001	0.584 ±0.000	0.405 ±0.002	0.493 ±0.008	0.596 ±0.001	0.930 ±0.000
	Peekaboo without the leash (KL 0)	0.539 ±0.006	0.584 ±0.000	0.403 ±0.001	0.463 ±0.009	0.586 ±0.009	0.930 ±0.001
	Peekaboo, encoder unfrozen	0.572 ±0.007	0.558 ±0.011	0.559 ±0.005	·	·	·
	Peekaboo, encoder unfrozen + switch	0.555 ±0.019	0.579 ±0.017	0.559 ±0.005	·	·	·
	Peekaboo, encoder unfrozen, no honesty fix	0.572 ±0.007	0.558 ±0.011	0.559 ±0.005	·	·	·
	pooled Peekaboo (one add-on, all suites' feedback)	0.538 ±0.002	0.569 ±0.007	0.398 ±0.006	0.475 ±0.004	0.594 ±0.002	0.930 ±0.000
	pooled Peekaboo + switch	0.542 ±0.000	0.569 ±0.007	0.401 ±0.000	0.475 ±0.004	0.597 ±0.000	0.930 ±0.000
	pooled Peekaboo without the honesty fix	0.538 ±0.002	0.569 ±0.007	0.398 ±0.006	0.475 ±0.004	0.594 ±0.002	0.930 ±0.000
calibration error (ECE)							
Vega 0.8B	base model	0.154	0.173	0.105	0.063	0.033	0.021
	Peekaboo	0.203 ±0.007	0.092 ±0.018	0.055 ±0.015	0.077 ±0.019	0.070 ±0.025	0.016 ±0.004
	Peekaboo + switch	0.184 ±0.021	0.152 ±0.003	0.055 ±0.015	0.063 ±0.001	0.070 ±0.025	0.016 ±0.004
	Peekaboo without the honesty fix	0.328 ±0.018	0.267 ±0.008	0.241 ±0.010	0.198 ±0.019	0.251 ±0.063	0.075 ±0.003
	Peekaboo without the leash (KL 0)	0.213 ±0.016	0.097 ±0.014	0.082 ±0.024	0.052 ±0.006	0.048 ±0.009	0.018 ±0.003
	pooled Peekaboo (one add-on, all suites' feedback)	0.193 ±0.024	0.108 ±0.034	0.066 ±0.009	0.062 ±0.031	0.068 ±0.023	0.019 ±0.005
	pooled Peekaboo + switch	0.157 ±0.004	0.108 ±0.034	0.066 ±0.009	0.048 ±0.011	0.068 ±0.023	0.019 ±0.005
	pooled Peekaboo without the honesty fix	0.267 ±0.038	0.308 ±0.022	0.232 ±0.051	0.154 ±0.018	0.311 ±0.110	0.073 ±0.014
Vega 4B	base model	0.141	0.158	0.099	0.050	0.054	0.027
	Peekaboo	0.172 ±0.003	0.098 ±0.008	0.049 ±0.009	0.044 ±0.013	0.035 ±0.012	0.018 ±0.003
	Peekaboo + switch	0.150 ±0.013	0.128 ±0.005	0.064 ±0.026	0.044 ±0.013	0.035 ±0.012	0.027 ±0.000
	Peekaboo without the honesty fix	0.235 ±0.003	0.167 ±0.012	0.223 ±0.009	0.171 ±0.004	0.311 ±0.026	0.049 ±0.016
	Peekaboo without the leash (KL 0)	0.182 ±0.011	0.099 ±0.005	0.075 ±0.027	0.049 ±0.016	0.052 ±0.004	0.016 ±0.002
	pooled Peekaboo (one add-on, all suites' feedback)	0.176 ±0.015	0.134 ±0.038	0.057 ±0.019	0.050 ±0.005	0.044 ±0.009	0.016 ±0.006
	pooled Peekaboo + switch	0.141 ±0.000	0.134 ±0.038	0.084 ±0.021	0.052 ±0.002	0.051 ±0.004	0.027 ±0.000
	pooled Peekaboo without the honesty fix	0.227 ±0.020	0.261 ±0.024	0.253 ±0.027	0.202 ±0.013	0.321 ±0.006	0.056 ±0.005
Laya	base model	0.212	0.113	0.174	0.183	0.306	0.012
	Peekaboo	0.157 ±0.004	0.063 ±0.012	0.071 ±0.006	0.052 ±0.005	0.086 ±0.014	0.018 ±0.004
	Peekaboo + switch	0.212 ±0.000	0.084 ±0.007	0.174 ±0.000	0.052 ±0.005	0.306 ±0.000	0.015 ±0.005
	Peekaboo without the honesty fix	0.319 ±0.009	0.228 ±0.002	0.329 ±0.027	0.351 ±0.014	0.347 ±0.012	0.059 ±0.000
	Peekaboo without the leash (KL 0)	0.178 ±0.015	0.066 ±0.015	0.065 ±0.008	0.056 ±0.013	0.088 ±0.018	0.018 ±0.002
	Peekaboo, encoder unfrozen	0.152 ±0.006	0.087 ±0.012	0.062 ±0.009	·	·	·
	Peekaboo, encoder unfrozen + switch	0.189 ±0.033	0.092 ±0.012	0.062 ±0.009	·	·	·
	Peekaboo, encoder unfrozen, no honesty fix	0.318 ±0.009	0.293 ±0.008	0.244 ±0.032	·	·	·
	pooled Peekaboo (one add-on, all suites' feedback)	0.164 ±0.007	0.096 ±0.019	0.067 ±0.015	0.052 ±0.016	0.087 ±0.006	0.016 ±0.003
	pooled Peekaboo + switch	0.212 ±0.000	0.096 ±0.019	0.174 ±0.000	0.052 ±0.016	0.306 ±0.000	0.012 ±0.000
	pooled Peekaboo without the honesty fix	0.328 ±0.033	0.271 ±0.029	0.349 ±0.054	0.328 ±0.036	0.364 ±0.019	0.056 ±0.009

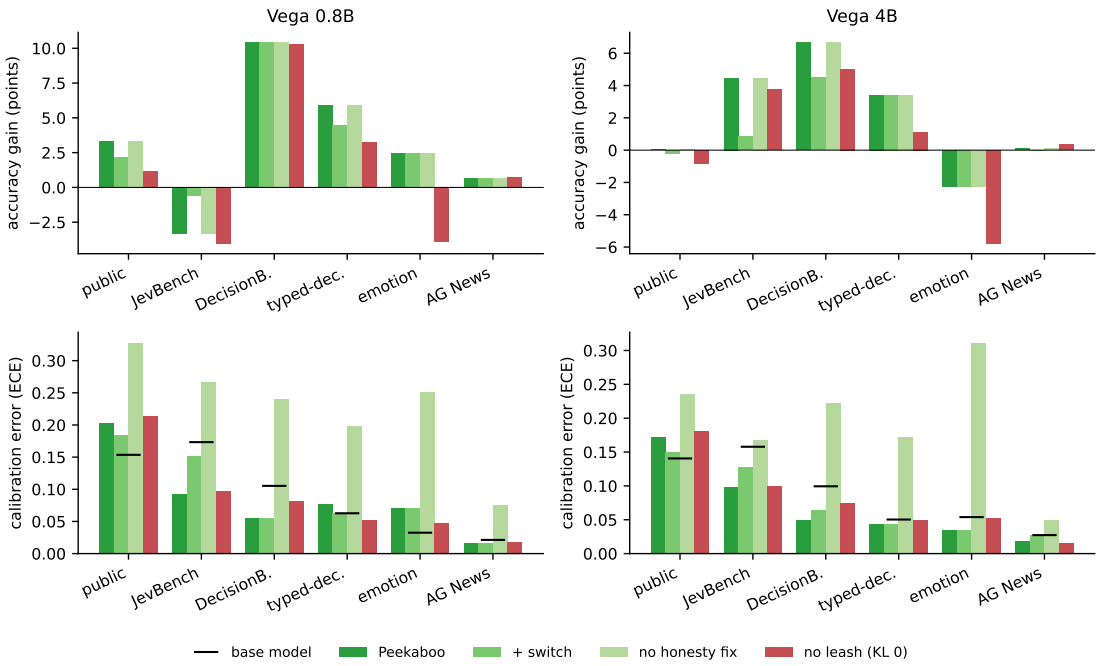


Figure 4: Ablations on Vega 0.8B (left) and 4B (right): accuracy gain (top) and ECE (bottom; black ticks are the base model). Removing the leash (KL 0) costs accuracy on five of six benchmarks; removing the honesty fix leaves accuracy unchanged but multiplies ECE; the switch trades gains for safety.

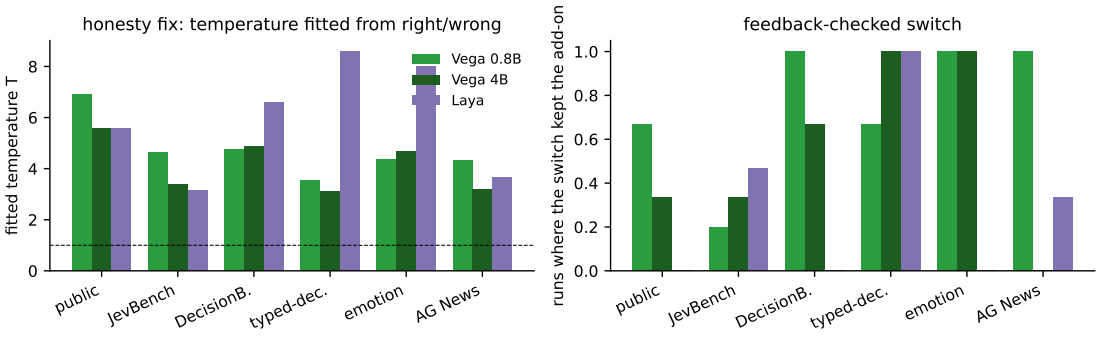


Figure 5: Left: the temperature fitted from right / wrong feedback, per model and benchmark (dashed line: $T = 1$). Values of 3 to 9 show how much the trained policy had to be softened. Right: the fraction of runs in which the feedback-checked switch kept the add-on.

how often it fired; a deployment with more calibration rows than JevBench's 42 would get a more reliable switch.

6.4 Pooled Peekaboo

One add-on trained on the feedback of every benchmark (Figure 6) is a mixed bag. At 0.8B it is slightly better on the public menu (+4.0 vs. +3.4) and AG News (+1.1 vs. +0.7), slightly worse on DecisionBench, typed-decisions and emotion, and it removes the JevBench loss (−0.1 vs. −3.3) although it never saw a JevBench row: feedback from 10,000 unrelated decisions teaches something general about how Vega misweighs options. At 4B, which already wins JevBench on its own, pooling is a little worse almost everywhere

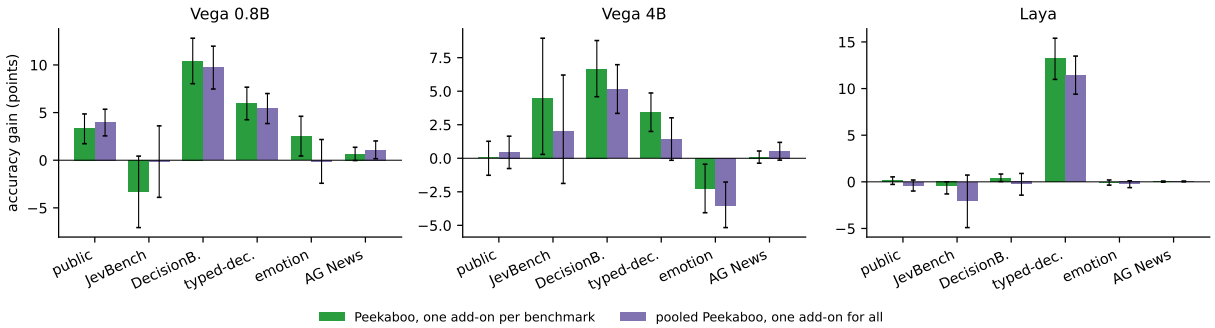


Figure 6: One add-on per benchmark (green) against one pooled add-on trained on the feedback of all five benchmarks with a training split (purple). For the pooled add-on JevBench is pure transfer.

(+2.0 vs. +4.5 on JevBench, +5.1 vs. +6.7 on DecisionBench). On Laya's frozen-encoder head it is uniformly slightly worse. We therefore present pooling as an option for a model that must stay good on tasks it gets no feedback on, not as the default.

6.5 Laya: how much of the model Peekaboo may change

Laya's own improvement is a whole-model fine-tune on labels, and on its home benchmark it is very strong: typed-decisions goes from 0.361 to 0.762 (+40.2; the README says 0.766). Peekaboo on the frozen-encoder head gets +13.2 [+11.0, +15.4] from right / wrong alone, our answer-key head +18.3: the head is the bottleneck, not the signal. Unfreezing the encoder under the same right / wrong signal gives ? [?] on typed-decisions (accuracy ?; Table 5, Figure 7): more than the answer-key head, still short of the fine-tune's +40.2, which sees $\log_2 k$ bits per row where Peekaboo sees one. Away from home the picture reverses. Laya's fine-tune loses 5.2 points on JevBench and gains little elsewhere (+0.9 public, +3.7 DecisionBench), the classic cost of specialising a whole model; the head-only Peekaboo changes nothing outside typed-decisions (±0.4); and the encoder-unfrozen Peekaboo gains +3.0 [+1.7, +4.4] on the public menu (0.572 vs. the fine-tune's 0.551) and +15.8 [+13.2, +18.7] on DecisionBench (0.559 vs. 0.438), with −3.0 on JevBench, ? on emotion and ? on AG News. Against the fine-tune directly it is +2.1 (public), +2.2 (JevBench), +12.1 (DecisionBench) and ? (typed-decisions) points. Table 7 breaks typed-decisions down by workflow.

Table 7: typed-decisions accuracy per workflow (500 test decisions each).

model	method	agent traces	customer service	invoices	security
Vega 0.8B	base	0.698	0.662	0.652	0.650
	TTT head	0.690	0.716	0.800	0.678
	answer-key adapter	0.703	0.743	0.805	0.687
	Peekaboo	0.705	0.731	0.761	0.703
Vega 4B	base	0.688	0.800	0.804	0.626
	TTT head	0.700	0.800	0.808	0.706
	answer-key adapter	0.707	0.813	0.815	0.763
	Peekaboo	0.695	0.802	0.801	0.757
Laya	base	0.388	0.380	0.360	0.316
	laya-typed-decisions	0.732	0.766	0.810	0.742
	answer-key head	0.507	0.501	0.619	0.551
	Peekaboo, head only	0.437	0.473	0.535	0.528

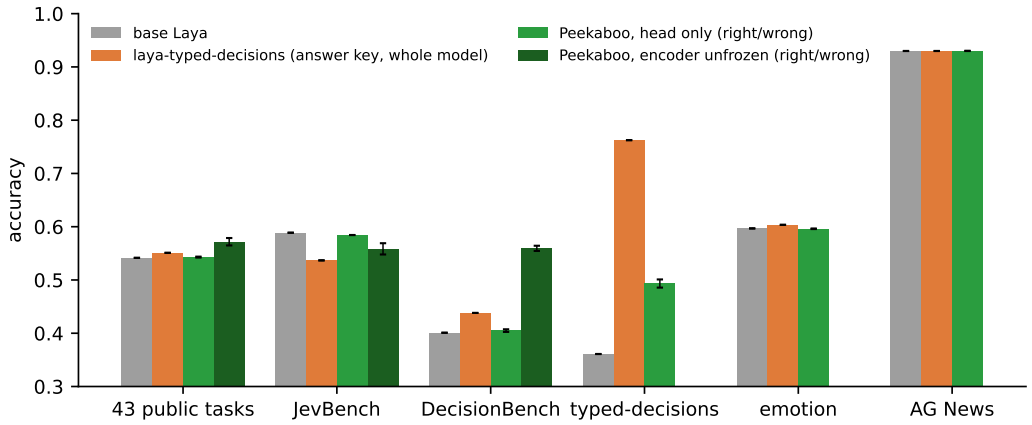
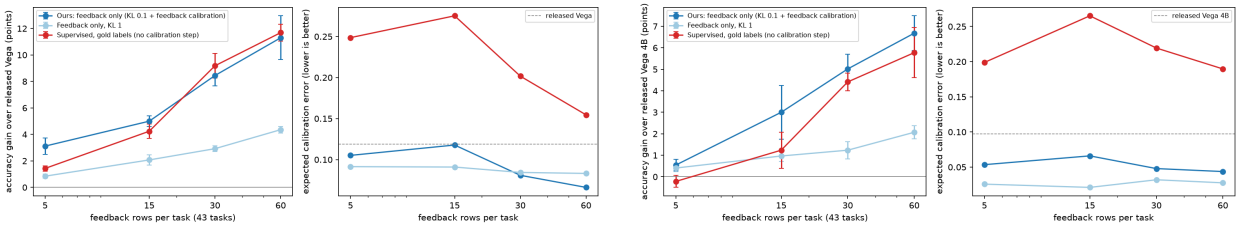


Figure 7: Laya accuracy on the six benchmarks: base, Laya's released answer-key fine-tune of the whole model, Peekaboo on the decision head alone, and Peekaboo with the encoder unfrozen.



(a) Vega 0.8B **(b)** Vega 4B

Figure 8: DecisionBench, 1,720 held-out rows: accuracy gain (left of each panel) and ECE (right) against the number of feedback rows per task. Dashed: base model.

6.6 How much feedback, one pass, unseen tasks

Our earlier DecisionBench study, which this grid extends, measured how the gain grows with feedback on a fixed 1,720-row test set (Figure 8). At 0.8B the gain is +3.1, +5.0, +8.4 and +11.3 points with 5, 15, 30 and 60 feedback rows per task; at 4B, +0.5, +3.0, +5.0 and +6.7. From 15 rows per task on, feedback-only learning stays within about one point of label-supervised learning. Seeing each of the 2,560 rows exactly once, as in a live deployment, gives +6.9 (0.8B) and +3.3 (4B). Feedback on 33 tasks improves 10 tasks that gave none by +4.2 (0.8B) and +4.6 (4B) points. A no-engine control, the same feedback and leash on a bilinear scorer over the same frozen features, reaches only 0.26 accuracy at either size: Vega's zero-shot energies are what make one bit per row enough.

6.7 Reward during training

Figure 9 shows the mean reward of the sampled answers over the eight passes. On Vega it rises steadily on every benchmark with a training split (DecisionBench 0.8B: 0.50 to 0.73); on Laya's frozen-encoder head it moves only on typed-decisions, consistent with the accuracy results. Since the sampled answer is right less often than the argmax, the reward curve also bounds the cost of exploration in a live system: on the first pass the policy is right about as often as the base model's samples, and better from the second pass on.

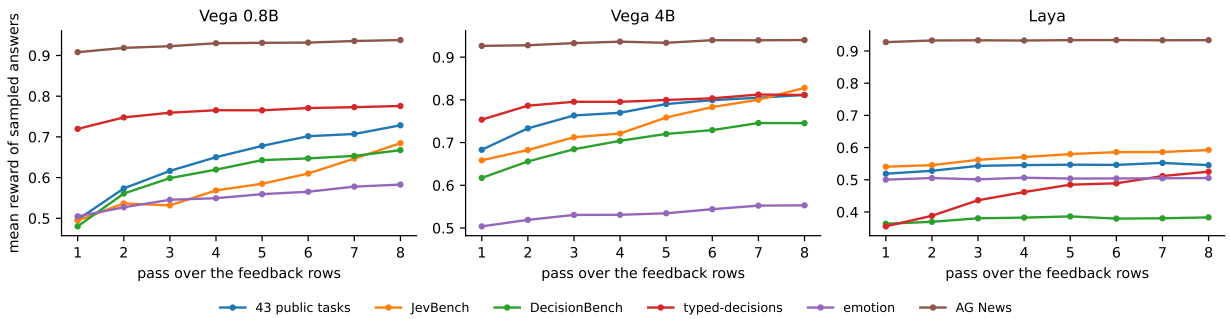


Figure 9: Mean reward of the sampled answers in each of the eight passes over the feedback rows (mean over seeds).

7 Discussion

When Peekaboo is the right tool. A model that is already deployed, collects right / wrong events as a by-product of use, and must stay good at everything it already does. There, with the same capacity, right / wrong recovers most of what the answer key would give (Section 6), the leash protects the rest, the switch guards against regressions, and the honesty fix keeps the probabilities usable. The gains are largest where the base model already leans towards the right option and mis-weighs it (DecisionBench's online-safety, home-automation and web domains in our earlier study), and smallest or negative where the task needs reasoning the base model lacks (DecisionBench's logic domain; JevBench's temporal-numeric family at 0.8B).

When it is not. A fixed task with thousands of fully labelled rows and permission to retrain the whole model. Laya's typed-decisions fine-tune (+40) is the clearest case; TTT on emotion is another. The answer key carries $\log_2 k$ bits per row against Peekaboo's one, and when the task is a clean single-label classification with many rows, that difference shows.

Why TTT loses on the public menu and DecisionBench. A TTT head is a per-task classifier with a fixed label set. Many DecisionBench rows have their own option list (a scale with different words per row, a list of candidate records), so `fit()` kept heads for 20 of the 43 tasks covering 394 of 1,075 rows and left the rest to the engine; Peekaboo changes the engine, which reads the option text, so it applies to every row. On the public menu, 50 labelled rows per task are enough for a logistic head to overturn Vega's zero-shot answer on tasks where the zero-shot answer was better.

8 Limitations

- **Simulated feedback.** The rewards are computed from gold answers: never wrong, never missing, never biased towards complaints. The method needs only a 0/1 signal, but we have not measured it under noisy, sparse or selective feedback. Our code has these settings (flip probability, report probability by outcome, logged off-policy feedback with clipped importance weights) and we leave their study to future work.
- **Exploration has a cost.** During learning the model samples from its probabilities instead of taking the argmax; Figure 9 bounds the cost but we did not design a conservative exploration policy (e.g. exploring only inside Vega's conformal set).
- **Hyper-parameters.** $\beta = 0.1$ was chosen in a pilot on parts of DecisionBench's 1,075 rows and never retuned; rank 8, 8 passes and learning rate 10^{-3} were fixed before any result. We did not tune TTT (VegaML defaults) or the answer-key add-on separately. Everything is reported with the same settings on all six benchmarks.

- **One temperature per benchmark.** On the 43-task public menu this is too coarse; a per-task or per-option-count temperature, fitted the same way, is an obvious next step.
- **Small benchmarks.** JevBench has 231 rows and 42-row calibration folds; its CIs are wide, and the switch is unreliable there.
- **Vega's typed-decisions.** Vega's README reports typed-decisions on six unpublished workflows; we could only use Laya's public release, so Vega's own typed-decisions number is not directly compared.

9 Conclusion

A deployed decision model can improve from the one bit of feedback it actually receives. On the benchmarks Vega's and Laya's authors chose, a small add-on trained by REINFORCE on right / wrong rewards, held on a KL leash and re-calibrated with a temperature fitted from the same feedback, beats VegaML's own answer-key method on four of six benchmarks at both model sizes, matches an answer-key adapter of the same capacity within one to two points, and keeps or improves calibration. On Laya it is far behind a whole-model answer-key fine-tune on that model's home benchmark but does not pay that fine-tune's price elsewhere, and with the encoder unfrozen it overtakes the fine-tune on the public tasks and DecisionBench. The leash and the feedback-fitted temperature are both necessary, and a feedback-checked switch makes the add-on safe to turn on. Everything is reproducible from one script.

Acknowledgements. We thank the authors of VegaML and Laya for releasing their models, benchmark code and training recipes, which made a like-for-like comparison possible.

References

[1] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, 2017.

[2] Convai Innovations. Laya: fast, non-autoregressive System 1 decision engine with calibrated probabilities. https://huggingface.co/convaiinnovations/laya, 2026. Python package `laya` 0.3.23; checkpoints `convaiinnovations/laya` and `convaiinnovations/laya-typed-decisions`.

[3] Miroslav Dudík, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *Proceedings of ICML*, 2011.

[4] Bradley Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[5] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: model alignment as prospect theoretic optimization. In *Proceedings of ICML*, 2024.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of ICML*, 2017.

[7] Hanno Labs. DecisionBench. https://huggingface.co/datasets/Hanno-Labs/decision-bench, 2026.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of ICLR*, 2022.

[9] Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, Craig Ferguson, Agata Lapedriza, Noah Jones, Shixiang Gu, and Rosalind Picard. Way off-policy batch deep reinforcement learning of implicit human preferences in dialog. In *arXiv preprint arXiv:1907.00456*, 2019.

[10] Julia Kreutzer, Joshua Uyheng, and Stefan Riezler. Reliability and learnability of human bandit feedback for sequence-to-sequence reinforcement learning. In *Proceedings of ACL*, 2018.

[11] Harold W. Kuhn. A simplified two-person poker. In *Contributions to the Theory of Games*, volume 1, pages 97–103, 1950.

[12] John Langford and Tong Zhang. The epoch-greedy algorithm for multi-armed bandits with side information. In *Advances in Neural Information Processing Systems*, 2008.

[13] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web*, 2010.

[14] LocalLLaMA. typed-decisions. https://huggingface.co/datasets/LocalLLaMA/typed-decisions, 2026.

[15] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning into quantiles. In *Proceedings of AAAI*, 2015.

[16] NandhaKishorM. VegaML: typed decisions from a frozen language model and a small physics engine. https://github.com/NandhaKishorM/vegaml, 2026. Python package `vegaml` 0.8.0; models `nandakishorm/vega-08b-public-intents` and the 4B release.

[17] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

[18] John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, 1999.

[19] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[20] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

[21] Elvis Saravia, Hsien-Chi Toby Liu, Yen-Hao Huang, Junlin Wu, and Yi-Shin Chen. CARER: Contextualized affect representations for emotion recognition. In *Proceedings of EMNLP*, 2018.

[22] Artem Sokolov, Julia Kreutzer, Christopher Lo, and Stefan Riezler. Learning structured predictors from bandit feedback for interactive NLP. In *Proceedings of ACL*, 2016.

[23] Florian Standhartinger. JevBench: public tiers. https://huggingface.co/datasets/fstandhartinger/jevbench, 2026.

[24] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of ICML*, 2020.

[25] Adith Swaminathan and Thorsten Joachims. Counterfactual risk minimization: learning from logged bandit feedback. In *Proceedings of ICML*, 2015.

[26] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, et al. Smarter, better, faster, longer: a modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

[27] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[28] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, 2015.

A Reproducing the results

`experiments/peekaboo_remaining/run.py` runs the entire grid: it reads every benchmark once through every model and caches the backbone features and the official records, replays each base model against its official prediction call, then runs every method and seed and writes `output/TABLES.md`, which is the source of every table here (via `paper/make_figures.py`). Finished steps are skipped, so the run is resumable; failed steps are retried and listed in `output/STATUS.md`. `experiments/vega_policy/reproduce.py` runs the DecisionBench study of Section 6.6 (feedback amount, one pass, unseen tasks, latency, the no-engine control and the Kuhn-poker [11] experiment that started this work). A second person reproduced the 0.8B DecisionBench gain (+10.4 vs. our +10.2) and the released 4B accuracy (0.635 vs. the README's 0.633) on a different machine before the grid was run.

Checks. Replaying each frozen model on its cached inputs matches the official prediction call to 0 (Vega) and $\leq 5 \times 10^{-5}$ (Laya, one question per forward pass, as its own call runs, since padded batches move its probabilities by up to 0.02 under autocast). Base Vega 0.8B and 4B reproduce the README's public-menu accuracy (0.556 / 0.715 vs. 0.556 / 0.717); base Laya and `laya-typed-decisions` reproduce the README's typed-decisions accuracy (0.361 / 0.762 vs. 0.362 / 0.766).

B Per-task results on the public menu

Table 8 lists every public task. Peekaboo's largest gains over base Vega 0.8B are on tasks with many options or an unusual label set (prompt injections, superglue_cb, tweet_sentiment), and its losses are concentrated on tasks where the TTT head also loses (unfair_tos, clinc150_oos); on Laya, the encoder-unfrozen variant is what moves the long-tail intent tasks.

Table 8: Accuracy on each of the 43 public tasks (up to 60 test rows each; mean over seeds). "key" is the answer-key adapter / head, "Pk" is Peekaboo, "fine-tune" is laya-typed-decisions, "Pk enc" is Peekaboo with the encoder unfrozen. Bold: the best method for that model on that task.

task	Vega 0.8B				Vega 4B				Laya			
	base	TTT	key	Pk	base	TTT	key	Pk	base	fine-tune	Pk head	Pk enc
aegis_prompt_safety	0.50	0.58	**0.65**	0.64	0.60	0.60	**0.78**	0.73	0.37	0.43	0.44	**0.69**
ag_news	0.97	0.92	0.97	**0.98**	0.95	0.95	**0.96**	0.94	**0.98**	**0.98**	**0.98**	**0.98**
anli_r3	0.27	**0.45**	0.40	0.38	0.35	**0.50**	0.46	0.41	0.43	0.40	0.42	**0.45**
arc_challenge	0.47	0.35	0.46	**0.48**	0.78	0.77	0.76	**0.81**	0.28	0.37	0.28	**0.38**
arc_easy	**0.60**	0.35	0.52	0.57	0.92	0.85	0.93	**0.94**	0.52	**0.58**	0.52	0.57
banking77	0.68	0.68	**0.71**	0.63	0.73	0.73	0.72	**0.75**	**0.42**	0.40	0.41	0.33
boolq	0.77	0.62	0.67	**0.81**	0.82	0.85	0.82	**0.87**	0.87	0.85	**0.88**	**0.88**
casehold	0.35	0.35	**0.51**	0.45	0.55	0.42	**0.68**	0.65	0.37	0.38	0.37	**0.41**
climate_fever	0.38	0.40	0.46	**0.49**	0.45	0.37	0.46	**0.50**	0.33	**0.35**	0.33	0.29
clinc150_oos	**0.67**	**0.67**	0.56	0.62	**0.90**	**0.90**	0.72	0.80	0.00	**0.32**	0.00	0.00
commonsenseqa	0.35	0.15	**0.42**	0.36	0.53	0.48	0.59	**0.61**	**0.45**	0.43	**0.45**	0.43
cruxeval_output	0.65	0.48	**0.88**	0.78	0.85	0.92	0.92	**0.95**	**0.78**	0.73	0.78	0.78
dbpedia14	0.60	**0.85**	0.77	0.71	0.90	**0.97**	0.91	0.92	0.88	0.83	0.88	**0.89**
emotion	**0.55**	0.47	0.36	0.52	**0.65**	0.45	0.40	0.57	**0.57**	**0.57**	**0.57**	0.50
enron_spam	**1.00**	0.92	0.99	0.99	**1.00**	0.98	**1.00**	**1.00**	0.92	0.90	0.94	**0.94**
glue_rte	**0.63**	0.48	0.59	0.60	**0.88**	0.85	0.84	0.85	**0.83**	0.82	**0.83**	0.77
hellaswag	0.25	0.25	**0.26**	**0.26**	**0.65**	0.52	0.49	0.56	0.37	0.37	0.35	**0.42**
jailbreak_cls	**0.98**	**0.98**	0.98	**0.98**	**1.00**	0.98	0.99	0.99	**0.98**	**0.98**	**0.98**	**0.98**
ledgar	0.32	0.32	**0.62**	0.40	**0.65**	**0.65**	0.64	0.63	**0.25**	0.20	**0.25**	0.14
massive_intent_en	0.38	0.38	**0.49**	0.42	0.60	0.60	**0.63**	0.59	0.38	**0.43**	0.38	0.40
medmcqa	0.18	**0.28**	0.24	0.27	0.45	0.50	**0.53**	0.47	**0.35**	**0.35**	**0.35**	**0.35**
medqa_usmle	**0.29**	**0.29**	0.27	0.27	0.51	0.36	0.49	**0.53**	0.22	0.27	0.22	**0.28**
mmlu	**0.37**	0.22	0.29	0.32	**0.72**	0.42	0.61	0.65	0.27	**0.35**	0.27	0.32
mmlu_pro	0.20	0.13	0.14	**0.22**	0.38	0.18	**0.43**	0.41	0.18	0.20	0.19	**0.21**
openbookqa	**0.58**	0.22	0.52	**0.58**	**0.83**	0.63	0.73	0.77	0.42	**0.50**	0.42	0.49
prompt_injections	0.75	**0.96**	0.87	0.87	0.85	**0.96**	0.81	0.88	0.65	0.60	0.72	**0.78**
pubmedqa	0.13	**0.57**	0.48	0.54	0.40	**0.63**	0.61	0.57	0.53	0.55	0.53	**0.57**
race_middle	**0.60**	0.37	0.52	**0.60**	0.80	**0.88**	0.82	0.82	**0.58**	0.52	0.58	0.55
sciq	**0.73**	**0.73**	0.58	0.72	0.93	0.87	0.93	**0.94**	0.52	0.53	0.52	**0.59**
superglue_cb	0.67	0.67	0.53	**0.77**	0.82	**0.89**	0.78	0.80	0.87	0.87	**0.88**	0.81
toxic_chat_jailbreak	**0.98**	**0.98**	0.96	0.93	0.93	0.45	**0.95**	0.92	0.83	0.90	0.81	**0.92**
toxic_chat_toxicity	**0.95**	0.93	0.91	0.91	**1.00**	0.98	0.98	0.98	0.97	0.97	**0.98**	0.90
trec_coarse	0.53	**0.72**	0.71	0.43	0.77	**0.88**	0.82	0.75	0.53	0.53	0.52	**0.71**
truthfulqa_mc1	0.36	0.36	**0.81**	0.75	0.62	0.62	**0.75**	**0.75**	0.16	0.24	0.16	**0.43**
tweet_hate	**0.65**	0.63	0.58	0.62	**0.68**	0.60	0.54	0.50	**0.73**	0.72	0.72	0.62
tweet_irony	**0.78**	0.62	0.71	0.74	**0.78**	0.67	0.68	0.72	**0.78**	0.74	0.75	0.68
tweet_offensive	0.78	0.80	0.80	**0.81**	0.78	0.81	0.80	**0.84**	0.69	0.71	0.67	**0.79**
tweet_sentiment	0.44	0.41	0.45	**0.54**	**0.58**	**0.58**	0.50	0.49	0.56	0.49	**0.58**	0.54
tweet_stance_abortion	0.36	0.47	**0.48**	0.44	**0.53**	0.49	0.47	0.42	0.33	0.31	0.33	**0.45**
unfair_tos	**0.74**	0.63	0.56	0.53	**0.88**	0.74	0.78	0.76	**0.88**	0.84	0.83	0.70
vitaminc	**0.47**	0.28	0.42	0.39	**0.62**	0.57	0.53	0.61	0.43	0.32	0.46	**0.61**
winogrande	0.50	0.50	0.48	**0.54**	0.57	0.57	**0.62**	0.61	0.53	0.48	0.53	**0.57**
yelp_stars	0.52	0.52	**0.54**	0.50	**0.57**	0.48	0.47	0.52	0.28	0.37	0.28	**0.50**

C Hyper-parameters

add-on, Vega	gated LoRA, rank 8, on the engine's input layers (`vegaml.attach_adapter`); 409k / 857k parameters
add-on, Laya	trainable copy of type embedding, 2-layer decision head and scorer; encoder frozen, or a trained copy ("encoder unfrozen")
optimiser	Adam; lr 10^{-3} (Vega adapter), 10^{-4} (Laya head), 2.5×10^{-5} (Laya encoder); grad-norm clip 1.0
schedule	8 passes, batches of 32 (Laya encoder: micro-batches of 8, one update per 32 rows)
reward	$r = \mathbf{1}[\text{sampled answer} = \text{gold}]$; baseline $b \leftarrow 0.9\, b + 0.1\, \bar{r}$, $b_0 = 0.5$
leash	$\beta = 0.1$ (ablation: 0)
honesty fix	T on a 61-point log grid in $[0.25, 32]$, maximising the Bernoulli likelihood of right / wrong on the calibration rows
splits	feedback rows 80 / 20 train / calibration per task by a fixed hash; JevBench 5-fold by paraphrase group
seeds	0, 1, 2